

系统程序设计 — Lab 1

UNIX I/O 性能实验与高吞吐量 I/O 服务器设计

Lab Document v2.0

1 实验概述

本实验分为两个部分，旨在帮助同学们深入理解 UNIX/Linux 下的 I/O 子系统，并培养实际动手能力和学术写作能力。

部分	内容	提交形式
Part A	UNIX read / stdio / fread / write 性能对比实验	源代码 + 实验报告 (PDF/Word)
Part B	poll / 异步 IO / 多线程 IO 对比与高吞吐量 IO 服务器设计	源代码 + 论文 (PDF)

2 Part A: UNIX I/O 基础性能对比

2.1 实验目标

通过实验测量和分析，理解以下几种 I/O 方式的性能差异及其背后的原因：

- 系统调用 `read()` 在不同 `BUFSIZE` 下的读取效率
- 标准 I/O 库 (stdio) 逐字符函数 `getc()` / `fgetc()` 的读取效率
- 标准 I/O 库块读取函数 `fread()` 在不同 `BUFSIZE` 下的读取效率
- 自己实现的 `my_fread()` 函数的效率表现
- `write()` 在有无 `O_SYNC` 标志下的写入效率差异

2.2 实验内容

2.2.1 测试 `read()` 的 I/O 效率

使用系统调用 `read()` 读取一个较大的文件（建议生成 500MB 以上的测试文件），分别以不同的 `BUFSIZE` (1、2、4、8、16、32、64、128、256、512、1024、2048、4096、8192、16384、32768、65536、16777216 字节) 进行读取。记录每种 `BUFSIZE` 下的实时时间 (wall clock)、用户时间和系统时间。

提示： 可以通过 `dd if=/dev/urandom of=testfile bs=1M count=500` 生成测试文件，使用 `clock_gettime()` 或 `gettimeofday()` 等函数计时。

2.2.2 测试 stdio (getc/fgetc) 的 I/O 效率

使用标准 I/O 库的逐字符读取函数 `getc()` 或 `fgetc()` 读取同一个测试文件，记录实时时间、用户时间和系统时间。思考：stdio 库内部维护了用户态缓冲区，为什么逐字符调用 `getc()` 的性能仍然远优于逐字节调用 `read()`？同时关注 `getc()` 宏实现与 `fgetc()` 函数实现之间是否存在性能差异。

2.2.3 测试 fread() 的 I/O 效率

使用标准库函数 `fread()` 读取同一个测试文件，分别以不同的 `BUFSIZE` 进行测试，记录相同的时间指标。对比 `fread()` 与 `read()` 和 `getc()` 的性能差异，分析 stdio 缓冲机制在其中起到的作用。

2.2.4 实现并测试 my_fread()

参考标准库 `fread()` 的工作原理，自己实现一个带缓冲区的读取函数 `my_fread()`。要求内部基于 `read()` 系统调用，维护一个用户态缓冲区，当缓冲区中的数据不足时才调用 `read()` 补充。将其性能与 `fread()` 进行对比分析。

2.2.5 测试 write() 的 I/O 效率 (无 O_SYNC)

使用 `write()` 写入数据到文件，不设置 `O_SYNC` 标志，测试不同 `BUFSIZE` 下的写入性能。

2.2.6 测试 write() 的 I/O 效率 (有 O_SYNC)

使用 `write()` 写入数据到文件，设置 `O_SYNC` 标志，测试不同 `BUFSIZE` 下的写入性能。对比有无 `O_SYNC` 的性能差异，并分析内核缓冲区的作用。

2.3 实验报告要求 (Part A)

实验报告可用中文或英文书写，应包含以下内容：

1. **实验环境说明**：操作系统、内核版本、文件系统类型、物理机/虚拟机、磁盘类型 (HDD/SSD) 等
2. **实验数据**：以表格形式展示各 `BUFSIZE` 下的时间数据 (real/user/sys)
3. **数据分析**：使用图表展示性能趋势，对比分析各种 I/O 方式的差异
4. **思考与讨论**：结合系统调用原理、内核缓冲机制、用户态缓冲等知识进行深入分析

⚠ 重要提示：请不要只罗列数据，要结合原理进行深入分析和思考。我们看重的是你的逻辑推理过程，而不是单纯的数据结果。

3 Part B: 高吞吐量 I/O 服务器设计与分析

3.1 背景与目标

在实际的服务器编程中，如何高效地处理大量并发的 I/O 请求是一个核心问题。从早期的阻塞式 I/O、到多路复用 (`select/poll/epoll`)、再到异步 I/O (`POSIX AIO / io_uring`) 和多线程模型，不同的 I/O 模型在不同场景下有着各自的优势与局限。

本部分要求同学们深入研究这些 I/O 模型，通过原理分析、实际实验和文献调研，以论文的形式提交一份关于高吞吐量 I/O 服务器设计的报告。

3.2 研究内容

3.2.1 I/O 模型原理分析

对以下 I/O 模型进行原理层面的分析和对比：

- 阻塞式 I/O (Blocking I/O)**：基本模型，说明其工作流程及局限性
- 多路复用 I/O (select/poll/epoll)**：分析 select 和 poll 的工作原理，重点分析 epoll 的边缘触发与水平触发模式，以及相对于 select/poll 的优势
- 异步 I/O (Asynchronous I/O)**：介绍 POSIX AIO 及 Linux 特有的 io_uring 机制，分析其与多路复用的本质区别
- 多线程 I/O**：分析线程池模型，讨论线程切换开销、锁竞争等问题

建议绘制各 I/O 模型的工作流程图（如时序图或状态转换图）以便更清晰地展示各模型的工作过程。

3.2.2 实验设计与实现

设计并实现一个简单的并发 I/O 测试程序，对比以下几种模型在处理并发连接时的性能表现：

- 基于 `poll()` 的 I/O 多路复用服务器
- 基于 `epoll` 的 I/O 多路复用服务器（可选，加分项）
- 基于 `io_uring` 的异步 I/O 服务器（可选，加分项）
- 基于多线程（线程池）的并发服务器

实验可以采用 TCP echo server 的形式，使用多个客户端并发连接进行压力测试。建议测量以下指标：

- 吞吐量 (Throughput)**：单位时间内处理的请求数
- 延迟 (Latency)**：单个请求的响应时间分布（平均值、P99 等）
- 并发连接数容量**：最大支持的并发连接数
- CPU 使用率及内存占用**

提示：可使用 `wrk`、`ab` (Apache Bench) 等工具进行压测，也可以自己编写客户端进行测试。

3.2.3 高吞吐量 I/O 服务器架构设计

基于前述的原理分析和实验结果，设计一个你认为最优的高吞吐量 I/O 服务器架构。要求包括：

- 整体架构图：包括线程模型、I/O 模型的选择及组合方式
- 关键设计决策的说明：为什么选择这种组合，有什么权衡
- 可以参考实际开源项目（如 Nginx、Redis、Netty 等）的设计思想

3.3 论文要求 (Part B)

Part B 要求以小论文的形式提交，可用中文或英文书写。论文应包含以下结构：

章节	内容说明
摘要	简要概述研究内容、方法和主要结论
引言	介绍研究背景、问题定义和研究动机
相关工作	调研现有的 I/O 模型文献及开源项目实践，可引用教材、论文或技术博客
原理分析	对各 I/O 模型的工作原理进行详细分析，配合图示说明
实验设计与结果	详细描述实验环境、测试方法、实验数据及图表分析

章节	内容说明
服务器架构设计	提出你的高吞吐量 I/O 服务器设计方案，包含架构图和设计理由
结论	总结研究发现，讨论局限性和未来改进方向
参考文献	列出引用的所有文献，格式规范（建议使用 IEEE 或 GB/T 7714 格式）

参考文献建议：

- W. Richard Stevens, Stephen A. Rago. *Advanced Programming in the UNIX Environment (APUE)*, 第14章
- W. Richard Stevens. *UNIX Network Programming*, Vol.1, 第6章 I/O Multiplexing
- Jens Axboe. *Efficient IO with io_uring* (2019)
- Nginx, Redis, libuv 等开源项目的架构文档

4 提交要求

4.1 提交内容

部分	源代码	报告/论文	备注
Part A	测试程序源码（含 Makefile）	实验报告（PDF 或 Word）	不需要可执行文件
Part B	服务器源码 + 测试客户端源码	论文（PDF）	建议附 README

4.2 目录结构

请将所有文件放在同一目录下，建议结构如下：

```
学号/
├─ partA/
│   ├─ main.c
│   ├─ Makefile
│   └─ report.pdf
├─ partB/
│   ├─ src/                # 服务器与客户端源码
│   ├─ Makefile
│   ├─ README.md
│   └─ paper.pdf
└─ README.md              # 总体说明
```

压缩为 学号姓名_系统程序设计lab1 后提交。

4.3 评分标准

评分项	分值	说明
Part A — 代码正确性	15分	代码能正确编译运行，实现完整
Part A — 实验报告	20分	数据完整、分析深入、有独立思考

评分项	分值	说明
Part B — 原理分析	20分	对 I/O 模型的原理阐述清晰、准确
Part B — 实验设计与数据	20分	实验设计合理，数据可信，分析到位
Part B — 服务器设计方案	15分	架构设计合理，论证充分
论文写作规范	10分	结构完整、表述清晰、参考文献规范

4.4 截止日期

请在截止日期前提交，迟交将扣分。具体截止日期请关注课程公告。

5 建议与提示

1. 实验环境建议使用物理机上的 Linux 系统，虚拟机可能引入额外的 I/O 层影响结果准确性。若使用虚拟机，请在报告中说明
2. 测试前清除操作系统缓存：`sync && echo 3 > /proc/sys/vm/drop_caches`，以减少缓存对结果的干扰
3. 多次运行取平均值，避免偶发因素影响结果
4. Part B 的实验部分不必追求极致性能，重点在于理解原理并能够有理有据地分析实验结果
5. 鼓励查阅 APUE 第3、5、14章及 UNP 第6章的内容
6. 如有任何问题，欢迎通过邮件联系助教

6 常见问题

Q：Part A 和 Part B 可以分开提交吗？

A：不可以，请将两部分一起打包提交。

Q：Part B 的实验是否必须实现所有四种 I/O 模型？

A：最低要求实现 poll 和多线程两种模型。实现 epoll 和异步 I/O 将作为加分项。

Q：论文的参考文献可以引用网络文章吗？

A：可以，但建议以教材和学术论文为主，网络文章为辅。引用时请注明 URL 和访问日期。

Q：可以用 C++ 写吗？

A：可以，但请确保使用的是 POSIX 系统调用接口，而不是 C++ 标准库的封装。